

Fine-Tuning Qwen 3.5-0.8B with LoRA: A Practical Guide

Domain-Specific LLM Training for Mineral Exploration

This tutorial walks through an end-to-end pipeline for fine-tuning a small large-language model (Qwen 3.5-0.8B) on domain-specific data related to mineral exploration. We use **LoRA** (Low-Rank Adaptation), a parameter-efficient method that adds tiny trainable adapters to a frozen base model, making it feasible to train on just ~150 examples without catastrophic overfitting.

Why this approach?

- Qwen 3.5-0.8B is small enough to fine-tune on a single GPU yet capable enough to produce useful answers.
- LoRA keeps the vast majority of model weights frozen, adding only 1-5M trainable parameters out of 800M total.
- Domain data (mineral exploration Q&A;) is scarce; parameter-efficient methods prevent memorization and preserve general knowledge.
- The pipeline covers data preparation, training, evaluation, and deployment.

Understanding the Data Pipeline

Two Data Types

The training data consists of two complementary formats:

- **QA (Direct Answer)** -- The model receives a question and produces a concise, factual answer with no visible reasoning.
- **CoT (Chain-of-Thought)** -- The model first reasons step-by-step inside `<think>...</think>` tags, then gives the final answer.

Raw vs Transformed Format

Raw data is stored as JSON objects with fields like `question`, `answer`, and optionally `reasoning`. The transformation step converts each example into the **chat-message format** expected by the model: a list of `{role, content}` dictionaries.

QA example (transformed):

```
[
  {"role": "system", "content": "You are a mineral exploration expert."},
  {"role": "user", "content": "What is pyrite?"},
  {"role": "assistant", "content": "Pyrite is an iron sulfide mineral..."}
]
```

CoT example (transformed):

```
[
  {"role": "system", "content": "You are a mineral exploration expert."},
  {"role": "user", "content": "Explain pyrite as a pathfinder mineral."},
  {"role": "assistant", "content":
    "<think>Pyrite often co-occurs with gold deposits because...\n"
    "So pyrite can serve as...\n"
    "Pyrite is a key pathfinder mineral for gold exploration..."}
]
```

Why `<think>` tags? Qwen 3.5 has a native thinking mode that uses these tags to separate internal reasoning from the final user-facing answer. By training with these tags, the model learns to produce structured reasoning that can be shown or hidden at inference time.

What is LoRA and Why Use It

LoRA (Low-Rank Adaptation) is a parameter-efficient fine-tuning technique. Instead of updating all 800 million parameters in Qwen 3.5-0.8B, LoRA freezes the original weights and injects small trainable matrices (called **adapters**) into specific layers of the transformer.

Key Parameters

Parameter	Meaning	Typical Value
r (rank)	Controls the size of the low-rank matrices. Higher = more capacity but more parameters.	ore 64
alpha	Scaling factor applied to LoRA updates. Effective scale = α / r .	16 - 128
Target modules	Which linear layers receive adapters (e.g., q_proj, v_proj, k_proj, o_proj).	Attention layers
Dropout	Dropout applied to LoRA layers to reduce overfitting.	0.05 - 0.1

Why LoRA for Small Datasets?

With only ~148 training examples, full fine-tuning would update all 800M parameters -- giving the model far too many degrees of freedom relative to the data. It would simply memorize the examples and fail to generalize. LoRA constrains the update to a low-rank subspace (typically 1-5M trainable parameters), acting as a strong implicit regularizer.

Comparison

	Full Fine-Tune	LoRA
Trainable params	~800M (100%)	~1-5M (<1%)
GPU memory	High (16+ GB)	Low (8-12 GB)
Overfitting risk	Very high with small data	Low
Training speed	Slower	Faster
Saved artifact	Full model copy (~1.6 GB)	Adapter files (~10-50 MB)

Training Configuration Explained

Batch Size & Gradient Accumulation

We cannot always fit a large batch into GPU memory, so we simulate it:

```
effective_batch_size = per_device_batch_size x gradient_accumulation_steps  
                     = 4 x 4 = 16
```

The model processes 4 examples at a time but accumulates gradients over 4 such micro-batches before performing a weight update. This gives the stability of a batch size of 16 with the memory cost of 4.

Learning Rate & Schedule

- **Base LR = 2e-4** -- This is the standard learning rate for LoRA fine-tuning. High enough to learn, low enough to avoid catastrophic forgetting.
- **Cosine schedule** -- After warmup, the learning rate follows a cosine curve, smoothly decaying to near-zero by the end of training. This helps the model settle into a stable minimum.

Warmup

The first 10% of training steps use a linearly increasing learning rate, starting from zero up to 2e-4. This prevents the randomly-initialized LoRA adapters from making large, destabilizing updates before the optimizer has built up momentum estimates.

bf16 (bfloat16 Mixed Precision)

We use bfloat16 rather than float32 for most computations. bfloat16 uses 16 bits but allocates 8 bits to the exponent (same as float32), giving it a much wider dynamic range than fp16. This prevents the overflow/underflow issues that plague fp16 training while cutting memory usage roughly in half.

Why Not QLoRA for Qwen 3.5?

QLoRA quantizes the base model to 4-bit before applying LoRA adapters. While this saves additional memory, the Qwen 3.5 documentation specifically warns against it: the 0.8B model is already small, and 4-bit quantization introduces higher quantization errors that degrade performance. Standard LoRA with bf16 is the recommended approach.

Key Metrics to Monitor

Training Loss

The primary optimization objective. It should decrease steadily over training steps. If it plateaus early, the learning rate may be too low. If it spikes or oscillates wildly, the learning rate is too high or there is a data quality issue.

Validation Loss

Measured on held-out data not used for training. It should decrease and then flatten. If validation loss begins to **increase** while training loss continues to decrease, the model is **overfitting**. Remedies: reduce epochs, increase LoRA dropout, or add more training data.

Gradient L2 Norm

Measures the magnitude of parameter updates at each step. A relatively smooth gradient norm indicates stable training. Sudden spikes suggest problematic examples or numerical instability. Gradient clipping (max_grad_norm) caps these spikes.

Learning Rate

Visualize the scheduled learning rate to verify the warmup phase ramps up linearly and the cosine decay brings it smoothly to near-zero.

Good vs Bad Loss Curves

Scenario	Train Loss	Val Loss	Diagnosis
Healthy	Smooth decrease	Decrease then flat	Model is learning well
Overfitting	Keeps decreasing	Decreases then rises	Too many epochs / too few data
Underfitting	High, barely moves	High, barely moves	LR too low / model too small
Unstable	Spikes & oscillates	Erratic	LR too high / bad data batch

Data Transformation Details

Step-by-Step Walkthrough of `prepare_data.py`

The data preparation script converts raw JSON files into the tokenizer-ready chat format. Here is the sequence of operations:

- **1. Load raw data** -- Read QA and CoT JSON files from the input directory.
- **2. Classify each example** -- Determine whether it is QA (no reasoning) or CoT (has a reasoning field).
- **3. Build chat messages** -- For each example, create a list of message dictionaries: system, user, and assistant.
- **4. Wrap CoT reasoning** -- For chain-of-thought examples, prepend the reasoning inside `<think>...</think>` tags before the final answer.
- **5. Shuffle the dataset** -- Randomize order to prevent order-dependent pattern learning.
- **6. Split into train/validation** -- Hold out a portion for monitoring overfitting during training.
- **7. Save as JSONL** -- Write each example as one JSON object per line.

How Chat Templates Work

The tokenizer's `apply_chat_template()` method converts the list of messages into a specific token sequence that the model was pretrained on. This includes special tokens like `<|im_start|>` and `<|im_end|>` that delimit each role's contribution. Using the correct template is critical -- a mismatched format means the model sees gibberish.

Why System Prompts Matter

The system message sets the model's persona and behavioral expectations. For our use case, it establishes the model as a mineral exploration expert. During training the model learns to condition its outputs on this prompt, so it should match what will be used at inference time.

Why We Shuffle

If all QA examples came first and all CoT examples second, the model might learn to associate early training with one style and late training with another. Shuffling ensures each batch contains a representative mix of both types, leading to more robust learning.

The Training Loop (Under the Hood)

Each training step performs the following sequence:

1. Forward Pass

Input tokens are fed through the transformer. Each layer computes attention and feed-forward operations. The LoRA adapters add their low-rank contributions to the frozen weight matrices. The output is a probability distribution over the vocabulary for each position.

2. Loss Computation

Cross-entropy loss is computed between the predicted next-token probabilities and the actual next tokens. Crucially, the loss is computed **only on assistant response tokens**, not on user or system tokens. This teaches the model to generate good answers without trying to learn how to generate questions.

3. Backward Pass

Gradients are computed via backpropagation. Because the base model weights are frozen, gradients only flow through and update the LoRA adapter matrices. This dramatically reduces the computation and memory required.

4. Optimizer Step (AdamW-8bit)

The AdamW optimizer updates LoRA weights using first and second moment estimates. The 8-bit variant stores optimizer states in 8-bit precision, cutting optimizer memory by ~75% with negligible impact on convergence. Weight decay (the 'W' in AdamW) applies L2 regularization to prevent weights from growing too large.

5. Gradient Checkpointing

Normally, all intermediate activations are stored during the forward pass so they can be used during backpropagation. Gradient checkpointing discards most of these activations and **recomputes** them during the backward pass. This trades ~30% more compute time for ~60% less memory, making it possible to train on longer sequences or with larger batch sizes.

Putting It Together

```
for each epoch:
    for each batch of 4 examples:
        logits = model(input_ids)           # forward pass
        loss = cross_entropy(logits, labels) # loss on assistant tokens only
        loss.backward()                     # compute gradients (LoRA only)
        if step % grad_accum == 0:         # every 4 micro-batches
            optimizer.step()                # update LoRA weights
            scheduler.step()                # adjust learning rate
            optimizer.zero_grad()           # reset gradients
```

After Training: Saving and Using the Model

LoRA Adapter Files

After training completes, the LoRA adapters are saved as small files (typically 10-50 MB). These contain only the trained low-rank matrices, not the full model. To use them, you load the base Qwen model and apply the adapter on top.

```
from peft import PeftModel
from transformers import AutoModelForCausalLM

base_model = AutoModelForCausalLM.from_pretrained("Qwen/Qwen3.5-0.8B")
model = PeftModel.from_pretrained(base_model, "./lora_adapters")
```

Merged Model

For simpler deployment, you can merge the LoRA weights back into the base model. This produces a single model that behaves identically but does not require the PEFT library at inference time.

```
merged = model.merge_and_unload()
merged.save_pretrained("./merged_model")
tokenizer.save_pretrained("./merged_model")
```

Inference Example

Once the model is loaded (adapter or merged), generating predictions follows the standard Hugging Face pattern:

```
from transformers import AutoTokenizer, AutoModelForCausalLM

tokenizer = AutoTokenizer.from_pretrained("./merged_model")
model = AutoModelForCausalLM.from_pretrained("./merged_model")

messages = [
    {"role": "system", "content": "You are a mineral exploration expert."},
    {"role": "user", "content": "What minerals indicate gold proximity?"}
]

input_ids = tokenizer.apply_chat_template(
    messages, return_tensors="pt", add_generation_prompt=True
)
output = model.generate(input_ids, max_new_tokens=512, temperature=0.7)
print(tokenizer.decode(output[0], skip_special_tokens=True))
```

Deployment Considerations

- **Adapter approach:** Smaller storage, can swap adapters for different domains, requires PEFT at runtime.
- **Merged approach:** Larger storage (~1.6 GB), simpler deployment, no PEFT dependency, slightly faster inference.

Evaluation Strategy

Why Evaluation Matters

Fine-tuning can cause a model to memorize training data, hallucinate domain facts, or lose general-purpose capabilities. A structured evaluation ensures the model has genuinely learned useful domain knowledge while remaining reliable.

The Scoring Rubric

Each model response is scored on five dimensions:

Criterion	What It Measures	Weight
Technical Accuracy	Are the geological/mineralogical facts correct?	High
Reasoning Quality	Is the chain-of-thought logical and coherent?	Medium
Completeness	Does the answer address all parts of the question?	Medium
Specificity	Does it provide concrete details, not vague generalities?	Medium
Hallucination Control	Does it avoid fabricating information?	High

Hallucination Traps

The evaluation includes deliberately tricky questions designed to catch the model making things up:

- Questions about fictional minerals or deposits that do not exist -- the model should say it does not know.
- Questions that mix real and fake information -- the model should identify and reject the false parts.
- Questions outside the training domain -- the model should acknowledge the limits of its expertise.

Base vs Fine-Tuned Comparison

The same set of evaluation questions is posed to both the original Qwen 3.5-0.8B base model and the fine-tuned version. Responses are scored using the rubric above, and the results are compared side-by-side. This quantifies the improvement from fine-tuning and reveals any regressions in general capability.

Common Issues and Troubleshooting

Loss Not Decreasing

Check that the data format matches the chat template exactly. Verify the learning rate is not too low (try $2e-4$ to $5e-4$). Ensure the loss is computed on the correct tokens (assistant only). Inspect a few training examples to rule out data corruption.

Overfitting (Validation Loss Increasing)

Reduce the number of training epochs (start with 3). Increase LoRA dropout (e.g., from 0.05 to 0.1). Add more training examples if possible. Reduce the LoRA rank to further constrain capacity.

Out of Memory (OOM)

Reduce `per_device_train_batch_size` (try 2 or 1). Reduce `max_seq_length` (e.g., from 2048 to 1024). Enable gradient checkpointing if not already on. Use bf16 to halve activation memory. Reduce LoRA rank.

Model Repeating Itself

This is usually an inference-time issue, not a training problem. Use `repetition_penalty` (e.g., 1.1-1.2) during generation. Adjust temperature (0.6-0.8) and `top_p` (0.9). If it persists, check that the chat template terminates correctly with an end-of-turn token.

Model Forgot General Knowledge (Catastrophic Forgetting)

Too many training epochs or too high a learning rate caused the model to overwrite its pretrained knowledge. Remedies: use fewer epochs (1-3), lower the learning rate, or reduce the LoRA rank. Consider mixing in a small amount of general-purpose data during training.

Quick Reference Checklist

- Verify data format with a manual inspection before training.
- Start with conservative hyperparameters (3 epochs, $lr=2e-4$, $r=16$).
- Monitor validation loss -- stop if it rises for 2+ evaluations.
- Save checkpoints frequently so you can roll back.
- Always compare against the base model to quantify improvement.
- Test with hallucination traps before deploying.